23CSR306 JAVA PROGRAMMING

Assignment – II

1. Assignment Question

Design and implement a console-based CRM application to manage Contacts, Leads, and Customers. Build it using Object-Oriented Programming principles in Java.

Requirements:

1. Create at least 4 classes:
    - o User – internal CRM user with id, name, role, email.
    - o Lead – potential customer with leadId, company, contactPerson, status, score.
    - o Customer – converted lead with customerId, company, primaryContact, contracts.
    - o CRMManager – service that manages users, leads, customers, and conversions.
2. Each class must include:
    - o ≥4 instance/static variables.
    - o A constructor to initialize values.
    - o ≥5 methods (getters/setters, addLead(), convertLead(), search(), listByStatus()).
3. Demonstrate OOPS Concepts:
    - o Inheritance → SalesManager extends User with extra approval privileges.
    - o Method Overloading → search() by name/company/phone.
    - o Method Overriding → custom toString() or display() in Lead vs Customer.
    - o Polymorphism → handle User and SalesManager through a User reference.
    - o Encapsulation → private fields with getters/setters.
4. Write a Main class (CRMAppMain) to test:
    - o Add users and leads.
    - o Convert a lead to a customer.
    - o Search/filter leads by status or score.
    - o Generate reports (leads per owner, conversions this week).

2. Java Source Code

CRMAppMain.java

```
package crm;
import java.util.*;
import java.time.*;
```

1. USER CLASS

```
class User {
```

```
 private int id;
 private String name;
 private String role;
 private String email;
 private String phone;

 public User(int id, String name, String role,
 String email, String phone) {
 this.id = id;
 this.name = name;
 this.role = role;
 this.email = email;
 this.phone = phone;
 }

 public int getId() { return id; }
 public String getName() { return name; }
 public String getRole() { return role; }
 public String getEmail() { return email; }
 public String getPhone() { return phone; }

 public void setName(String name) { this.name = name; }
 public void setRole(String role) { this.role = role; }
 public void setEmail(String email) { this.email = email; }
 public void setPhone(String phone) { this.phone = phone; }

 public void display() {
 System.out.println(this);
 }

 @Override
 public String toString() {
 return "User ID: " + id +
 ", Name: " + name +
 ", Role: " + role +
 ", Email: " + email +
 ", Phone: " + phone;
 }
}
```

2. SALESMANAGER CLASS - INHERITANCE

```
class SalesManager extends User {
 private double approvalLimit;
 private String department;
 private int teamSize;
 private boolean canApprove;

 public SalesManager(int id, String name, String email,
 String phone, double approvalLimit,
 String department, int teamSize) {
 super(id, name, "Sales Manager", email, phone);
```

```
this.approvalLimit = approvalLimit;
this.department = department;
this.teamSize = teamSize;
this.canApprove = true;
}

public double getApprovalLimit() { return approvalLimit; }
public void setApprovalLimit(double limit) { approvalLimit = limit; }
public String getDepartment() { return department; }
public void setDepartment(String department) { this.department = department; }
public int getTeamSize() { return teamSize; }
public void setTeamSize(int teamSize) { this.teamSize = teamSize; }
public boolean canApprove() { return canApprove; }

public boolean approveContract(double amount) {
return canApprove && amount <= approvalLimit;
}

@Override
public void display() {
System.out.println("Sales Manager: " + getName());
System.out.println("Department: " + department);
System.out.println("Approval Limit: " + approvalLimit);
System.out.println("Team Size: " + teamSize);
}

@Override
public String toString() {
return super.toString() +
", Department: " + department +
", Approval Limit: " + approvalLimit;
}
}
```

3. LEAD CLASS

```
class Lead {
private int leadId;
private int score;
private int ownerId;
private String company;
private String contactPerson;
private String status;
private String phone;
private LocalDate createdDate;

public Lead(int leadId, String company, String contactPerson,
String status, int score, int ownerId, String phone) {
this.leadId = leadId;
this.company = company;
this.contactPerson = contactPerson;
```

```
this.status = status;
this.score = score;
this.ownerId = ownerId;
this.phone = phone;
this.createdDate = LocalDate.now();
}

public int getLeadId() { return leadId; }
public int getScore() { return score; }
public int getOwnerId() { return ownerId; }
public String getCompany() { return company; }
public String getContactPerson() { return contactPerson; }
public String getStatus() { return status; }
public String getPhone() { return phone; }
public LocalDate getCreatedDate() { return createdDate; }

public void setScore(int score) { this.score = score; }
public void setStatus(String status) { this.status = status; }

public boolean isQualified() {
return score >= 70;
}

public void display() {
System.out.println(this);
}

@Override
public String toString() {
return "Lead ID: " + leadId +
", Company: " + company +
", Contact: " + contactPerson +
", Status: " + status +
", Score: " + score +
", Owner ID: " + ownerId;
}
}
```

4. CUSTOMER CLASS

```
class Customer {
private int customerId;
private int ownerId;
private String company;
private String primaryContact;
private String phone;
private List<String> contracts;
private LocalDate convertedDate;

public Customer(int customerId, int ownerId, String company,
String primaryContact, String phone) {
```

```
this.customerId = customerId;
this.ownerId = ownerId;
this.company = company;
this.primaryContact = primaryContact;
this.phone = phone;
this.contracts = new ArrayList<>();
this.convertedDate = LocalDate.now();
}

public int getCustomerId() { return customerId; }
public int getOwnerId() { return ownerId; }
public String getCompany() { return company; }
public String getPrimaryContact() { return primaryContact; }
public String getPhone() { return phone; }
public List<String> getContracts() {
return Collections.unmodifiableList(contracts);
}
public LocalDate getConvertedDate() { return convertedDate; }

public void setPrimaryContact(String contact) { primaryContact = contact; }

public void addContract(String contract) {
contracts.add(contract);
}

public void display() {
System.out.println(this);
}

@Override
public String toString() {
return "Customer ID: " + customerId +
", Company: " + company +
", Contact: " + primaryContact +
", Contracts: " + contracts;
}
}
```

5. CRM MANAGER CLASS

```
class CRMManager {
private List<User> users;
private List<Lead> leads;
private List<Customer> customers;
private int nextCustomerId;
private String organization;

public CRMManager(String organization) {
this.organization = organization;
users = new ArrayList<>();
leads = new ArrayList<>();
customers = new ArrayList<>();
```

```
nextCustomerId = 1;
}

public void addUser(User user) {
users.add(user);
System.out.println("User added successfully.");
}

public void addLead(Lead lead) {
leads.add(lead);
System.out.println("Lead added successfully.");
}

→ Search by contact person
public void search(String person) {
for (Lead lead : leads) {
if (lead.getContactPerson().equalsIgnoreCase(person)) {
System.out.println(lead);
}
}
}

→ Search by company
public void search(String company, int option) {
for (Lead lead : leads) {
if (lead.getCompany().equalsIgnoreCase(company)) {
System.out.println(lead);
}
}
}

// Search by phone number
public void search(long phone) {
String number = String.valueOf(phone);
for (Lead lead : leads) {
if (lead.getPhone().equals(number)) {
System.out.println(lead);
}
}
}

public void listByStatus(String status) {
for (Lead lead : leads) {
if (lead.getStatus().equalsIgnoreCase(status)) {
System.out.println(lead);
}
}
}

public void filterByScore(int minimumScore) {
for (Lead lead : leads) {
```

```
if (lead.getScore() >= minimumScore) {
System.out.println(lead);
}
}
}

public void convertLead(int leadId) {
for (Lead lead : leads) {
if (lead.getLeadId() == leadId &&
!lead.getStatus().equalsIgnoreCase("Converted")) {
Customer customer = new Customer(
nextCustomerId++,
lead.getOwnerId(),
lead.getCompany(),
lead.getContactPerson(),
lead.getPhone()
);
customers.add(customer);
lead.setStatus("Converted");
System.out.println("Lead converted successfully!");
System.out.println(customer);
return;
}
}
System.out.println("Lead not found or already converted.");
}

public void reportLeadsPerOwner() {
System.out.println("\n--- Leads Per Owner ---");
for (User user : users) {
int count = 0;
for (Lead lead : leads) {
if (lead.getOwnerId() == user.getId()) {
count++;
}
}
System.out.println(user.getName() + ": " + count);
}
}

public void reportConversionsThisWeek() {
System.out.println("\n--- Conversions This Week ---");
LocalDate today = LocalDate.now();
LocalDate weekStart = today.with(DayOfWeek.MONDAY);
int count = 0;
for (Customer customer : customers) {
LocalDate date = customer.getConvertedDate();
if (!date.isBefore(weekStart) && !date.isAfter(today)) {
count++;
}
}
```

```
System.out.println("Total conversions this week: " + count);
}

public void displayUsers() {
System.out.println("\n--- Users ---");
for (User user : users) {
user.display();
}
}

public void displayLeads() {
System.out.println("\n--- Leads ---");
for (Lead lead : leads) {
lead.display();
}
}

public void displayCustomers() {
System.out.println("\n--- Customers ---");
for (Customer customer : customers) {
customer.display();
}
}

public String getOrganization() {
return organization;
}
}
```

6. MAIN CLASS

```
public class CRMAppMain {
 public static void main(String[] args) {
 CRMManager crm = new CRMManager("ABC Organization");

 → Add users
 User user1 = new User(
 101, "Adithya", "Sales Executive",
 "adithya@gmail.com", "9876543210"
 );
 User user2 = new SalesManager(
 102, "Gowri", "gowri@gmail.com",
 "9876501234", 50000, "Sales", 5
 );
 crm.addUser(user1);
 crm.addUser(user2);

 → Polymorphism
 User reference = new SalesManager(
 103, "Kumar", "kumar@gmail.com",
 "9123456780", 75000, "Marketing", 4
```

```
);
reference.display();

→ Add leads
crm.addLead(new Lead(
1, "ABC Company", "Ravi",
"New", 85, 101, "9000011111"
));
crm.addLead(new Lead(
2, "XYZ Company", "Priya",
"Contacted", 60, 102, "9000022222"
));
crm.addLead(new Lead(
3, "PQR Company", "Karthik",
"New", 90, 101, "9000033333"
));

→ Display all leads
crm.displayLeads();

→ Search by contact person
System.out.println("\n--- Search by Contact ---");
crm.search("Ravi");

→ Search by company
System.out.println("\n--- Search by Company ---");
crm.search("XYZ Company", 1);

→ Search by phone number
System.out.println("\n--- Search by Phone ---");
crm.search(9000011111L);

→ Filter leads by status
System.out.println("\n--- New Leads ---");
crm.listByStatus("New");

→ Filter leads by score
System.out.println("\n--- Leads with Score >= 70 ---");
crm.filterByScore(70);

→ Convert lead into customer
System.out.println("\n--- Convert Lead ---");
crm.convertLead(1);

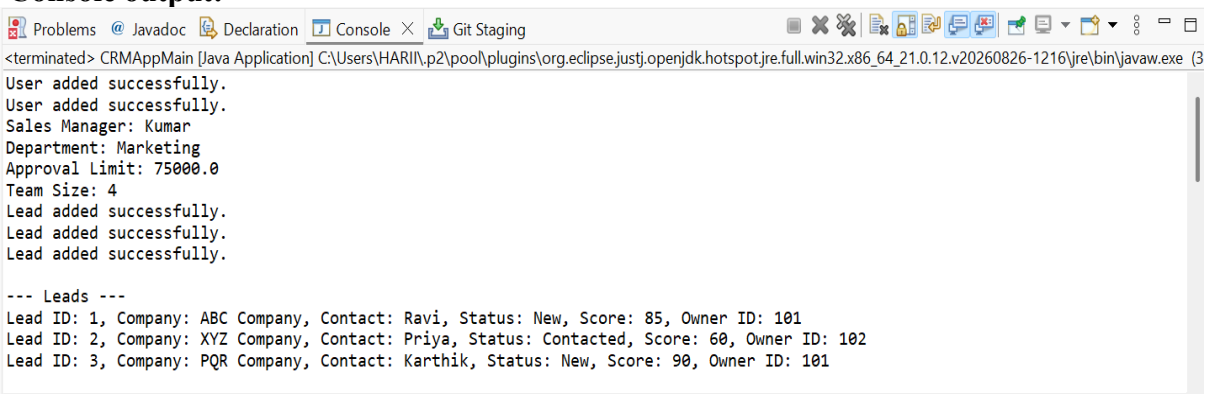
→ Display customers
crm.displayCustomers();

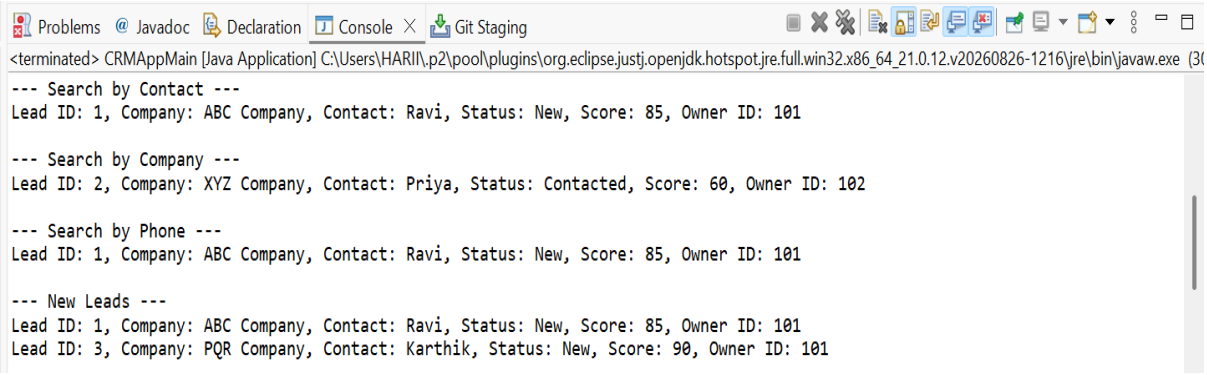
→ Generate reports
crm.reportLeadsPerOwner();
crm.reportConversionsThisWeek();
```

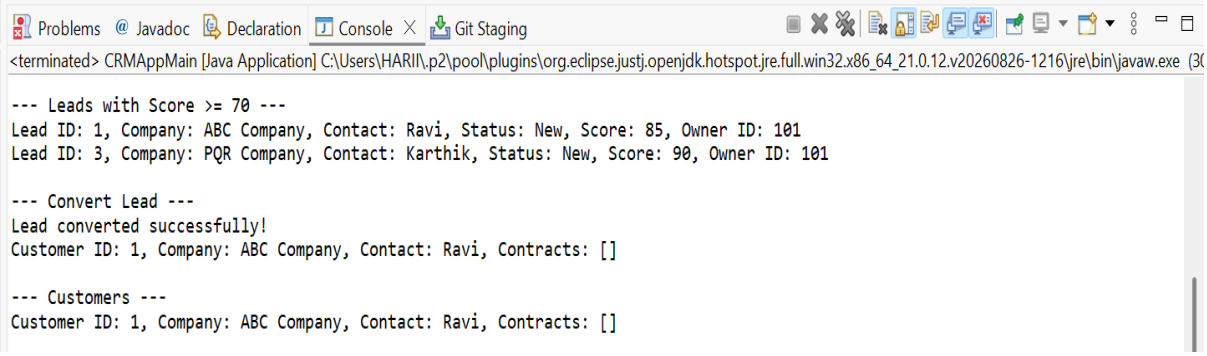
```
System.out.println("\nCRM Application Completed.");
 }
}
```

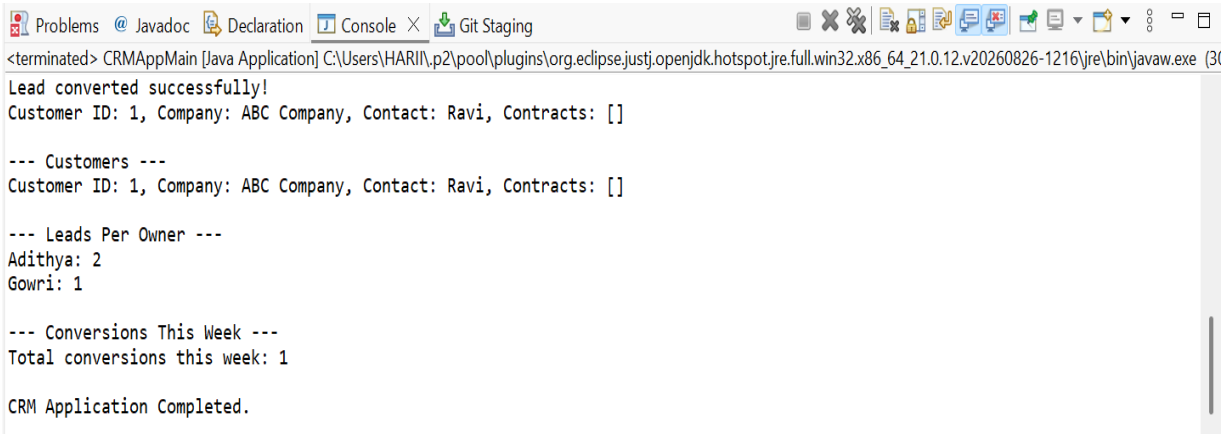
3. Execution Output Screenshots

Console output:









GitHub Repository Link:

https://github.com/ADITHYA-31/CustomerRelationship-Management-app-.git